TechBridge Sol

Engineering Policies and Best Practices

Effective Date: January 2026
Applies To: All Employees, Contractors, and Temporary Staff

1. Purpose

The purpose of this Engineering Policy is to define coding standards, version control procedures, documentation practices, and software development guidelines for TechBridge Sol. The goal is to ensure high-quality, maintainable, and secure code while supporting collaboration across teams and locations.

Compliance with these policies is mandatory for all developers, engineers, and contractors involved in software development.

2. Coding Standards and Best Practices

All engineers are expected to write code that is:

- **Readable:** Clear naming conventions, consistent formatting, and meaningful comments.
- **Maintainable:** Modular design, proper abstraction, and minimal duplication.
- **Testable:** Includes unit, integration, and automated tests where applicable.

- **Secure:** Follows secure coding practices, input validation, and handles sensitive data safely.

Language-specific conventions must be adhered to. For Python, follow **PEP 8** standards; for JavaScript, adhere to the company ESLint configuration.

Code reviews are required for all changes before merging to shared branches.

3. Version Control Policy

TechBridge Sol uses **Git** for version control. Key requirements:

- **Branching Model:**
  - `main` – Production-ready stable code
  - `develop` – Integration of features for testing
  - Feature branches – Short-lived, descriptive naming (`feature/xyz`)
  - Hotfix branches – For urgent production fixes
- **Commit Messages:**
  - Use clear, descriptive messages
  - Include issue/ticket reference if applicable
  - Examples:

```
feat(login): add two-factor authentication
fix(api): resolve token expiration bug
```

- **Pull Requests (PRs):**
  - PRs must include description, testing notes, and screenshots if needed
  - Require at least one reviewer approval
  - Merge only after CI/CD checks pass
- **Tagging and Releases:**
  - Use semantic versioning (`vX.Y.Z`)
  - Tags applied after approval of production release

4. Continuous Integration / Continuous Deployment (CI/CD)

All code changes are integrated, tested, and deployed using automated pipelines. Guidelines include:

- Every commit to `develop` triggers automated tests
- Code must pass all unit, integration, and linting checks before merging
- Deployment to production requires approvals from engineering leads and successful staging validation
- Rollback procedures must be documented and tested

CI/CD pipelines are version-controlled and maintained by the engineering team.

5. Documentation Guidelines

All engineering work must be documented to ensure knowledge sharing and maintainability:

- **Code Documentation:** Inline comments, docstrings, and type annotations
- **API Documentation:** Swagger/OpenAPI specifications or equivalent
- **Readme Files:** Each repository must include a README with setup instructions, dependencies, and usage
- **Internal Wiki:** High-level architecture diagrams, design decisions, and troubleshooting notes

Documentation should be updated concurrently with code changes.

6. Code Review and Quality Assurance

Code quality is maintained through mandatory reviews:

- Every PR must be reviewed by at least one senior engineer
- Reviewers check for adherence to coding standards, logic correctness, security concerns, and documentation completeness

- Automated quality gates include linting, test coverage thresholds, and static code analysis
- Feedback must be addressed promptly before merging

7. Security Practices in Development

Engineers must incorporate security throughout the development lifecycle:

- Follow **secure coding standards** (e.g., OWASP guidelines)
- Validate all user inputs and sanitize data
- Do not hardcode credentials, secrets, or API keys
- Store sensitive data securely using encryption and access control
- Report any discovered vulnerabilities immediately to the Security team

8. Testing Policy

All code must be tested rigorously before deployment:

- Unit tests for functions and modules
- Integration tests for cross-component interactions
- End-to-end tests for critical workflows
- Automated regression tests for production pipelines

Test coverage targets should be met and documented. Failures in tests must block merges until resolved.

9. Environment and Dependency Management

Engineering teams must maintain consistent development environments:

- Use version-controlled environment configuration files
- Use containerization (e.g., Docker) where applicable

- Dependencies must be reviewed, updated regularly, and approved for security compliance
- Avoid unnecessary or deprecated libraries

10. Policy Review and Updates

TechBridge Sol will review and update this Engineering Policy annually or as technology and business needs evolve. Engineers are expected to comply with the latest version. Managers are responsible for ensuring enforcement and awareness.